A 208 state machine for arithmetic consistency

A new contextual fold below the 209 state threshold

Research prepared for Vladimir Reshetnikov

29 October 2026

Abstract

We construct a binary, single-tape Turing machine with **208 working states and 416 transitions**, including its all-zero-tape initializer. The new reduction folds native state 172 into 211 in the exact supplied Compatibility 209-state machine. It changes two surviving transition rows and deletes two outgoing rows. No source reassociation, new tape symbol, additional tape, prewritten input, or omitted initialization is used.

The proof uses the different predecessor contexts of the two states. Four complete-tape identities replace one old transition by one, three, or seven new transitions; all 414 other old rows commute with the state projection. This gives a PA-formalizable halting equivalence independently of the numerical program. A fresh universal symbolic replay of the final literal table also proves all 5,084 guarded interpreter contracts, covers all 416 rows, and preserves the exact 48,140-step startup and its 141-step source bridge.

Composing the new local theorem with the retained parent calibration yields

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(U_{208}, \mathrm{blank}) \iff \mathrm{Con}(\mathsf{U}) \iff \mathrm{Con}(\mathrm{PA}).$$

This is an ordinary mathematical, PA-formalizable proof, not a newly exported proof-assistant certificate. It does not assert unconditional nonhalting or global state minimality. The six named incoming archives contain only two distinct rooted labelled machine classes, not six independent savings.

Primary artifact `machine/u208.tm`. Working states $0, \ldots, 207$, separate halt label 208; initial state and head coordinate 0; a two-way infinite binary tape initially all zero; left and right moves only. All $208 \cdot 2 = 416$ working-state/symbol pairs are defined.

What is new One additional contextual state fold, its complete local proof, the exact 208-state table, new independent checks, and the comparison of the current 209-state inputs. The proof-search program, unary theory and arithmetic calibration are inherited, with provenance retained separately.

1 The result and its exact identities

The requested strict threshold is met: $208 < 209$. Let P denote the Compatibility parent and C the new machine in native labels. Both have start state 0 and nonworking halt label 254. Sparse labels are names, not a count. The release dense table renames the 208 surviving working labels in ascending order to $0, \ldots, 207$, and 254 to 208. Its bijection and both literal tables are supplied and checked.

Property	Parent P	New machine C
Working states	209	**208**
Transition rows	418	**416**
Ordered source primitives	3,909	3,909
Maximum source address depth	41	41
Maximum consecutive zero address bits	6	6
Guarded primitive contracts	5,084	5,084
New startup transitions	0	0

The exact identities are SHA-256 hashes of the supplied bytes:

New native table
`bdf724e5a4c5d25567a8b51d50db8ff8b6da3e42b0f03a73794f470171fdcbbe`

New dense table
`5d487fb3ea29a84bedffec600af660ffeb8de3711838d67c41b01aec50c2a4ed`

Compatibility parent native table
`4564c4876f9b18725bfb3505ec1f50d1ebce572502f2c6381ad673ecfdc3c3e1`

Unchanged source map
`912724f29b0de28b3326c710fdf1f3863ebd07058c6db1a9b4bbaffb3ae5e3f3`

A row `q b w D r` reads b in state q, writes w, moves one cell in direction D, and enters r. Entry into the halt label ends the computation after that write and move. Every working state and the halt vertex are reachable in the labelled transition graph; this says nothing about whether halt is reached on the blank computation.

Theorem 1.1 (New local reduction)**.** *PA proves that P and C, started in state 0 on the same represented initial tape, halt equivalently. If they halt, their times satisfy*

$$T_P \le T_C \le 7T_P.$$

For the blank tape, the complete finite startup is unchanged, not merely bounded by this estimate.

The proof is given next without assuming consistency of any theory. The inherited endpoint enters only after this local machine theorem.

2 A contextual fold of native states 172 and 211

2.1 The entire literal edit

Define $\rho(172) = 211$ and let ρ fix every other state, including 0 and 254. Starting with the exact parent table, make only these edits:

Old row	Parent action	New action
116/0	$0R172$	$0R211$
211/0	$0R157$	$0L214$
172/0	$0L67$	deleted
172/1	$1R184$	deleted

The 211/1 row remains $1R184$. State 172 has just one incoming row, 116/0, so the first edit accounts for every redirected target. All other surviving rows are unchanged byte-for-byte after normalized serialization. In particular, the source map is literally unchanged.

2.2 The predecessor contexts are exhaustive

Put the head at coordinate 0 at the configuration being considered. The following facts hold at every occurrence in any old run started from state 0, irrespective of the exterior tape.

Lemma 2.1 (Context at 172)**.** *At old state 172, cells -2 and -1 contain 1 and 0 respectively.*

Proof. The only transition entering 172 is 116/0, which writes 0 and moves right. The only transition entering 116 is 163/1, which writes 1 and moves right. Neither 172 nor 116 is initial. These two immediate predecessors place the two claimed bits and do not overwrite them before arrival at 172. □

Lemma 2.2 (Context at 211)**.** *At old state 211, if the current bit is 0 then cell -1 contains 1.*

Proof. The complete incoming list is $216/0 : 0L211$ and $216/1 : 1R211$. The latter writes the required preceding 1 directly. In the former case, all incoming rows to 216 move right:

$$132/1 : 1R216, \qquad 199/1 : 1R216, \qquad 214/0 : 0R216.$$

If either of the first two was used, the next left move to 211 reads the just-written 1, contrary to the current-bit hypothesis. Hence the entry was 214/0. The only entry to 214 is $217/1 : 1R214$. Its written 1 is immediately left of the zero subsequently read by 211. All these states differ from the initial state, so no first-visit exception is possible. □

These are finite predecessor arguments about the old run. It is neither necessary nor correct to assume that every intermediate visit of the new run has the old meaning of its state label. Only block boundaries are projected old configurations.

2.3 Four complete tape identities

Write q_h for state q with head coordinate h. For an old 172/0 configuration the known bits are $(-2, -1, 0) = (1, 0, 0)$, and the new run is

$$\begin{aligned} 211_0 &\to 214_{-1} \to 216_0 \to 211_{-1} \\ &\to 214_{-2} \to 163_{-1} \to 157_0 \to 67_{-1}. \end{aligned}$$

This is seven transitions. The endpoint is exactly the old $0L67$ endpoint. For old 211/0, the known bits are $(-1, 0) = (1, 0)$ and

$$211_0 \to 214_{-1} \to 163_0 \to 157_1,$$

three transitions, with exactly the old $0R157$ endpoint. If either old state reads 1, the shared $1R184$ row gives the matching endpoint in one transition.

Old state	Read	Known cells	Endpoint	New steps
172	0	$(-2, -1, 0) = (1, 0, 0)$	67_{-1}	7
172	1	$(-2, -1, 0) = (1, 0, 1)$	184_1	1
211	0	$(-1, 0) = (1, 0)$	157_1	3
211	1	$(0) = (1)$	184_1	1

Every transition on these paths writes back the bit it reads. No cell outside the displayed known cells is read. Thus the equality is of complete tapes, not only a finite projection or a sampled exterior. No intermediate configuration is halting. All 414 parent rows outside the two exceptional states satisfy the one-step equation

$$C(\rho(q), b) = (w, D, \rho(r)) \quad \text{when } P(q, b) = (w, D, r).$$

The verifier enumerates those rows and checks each local path using a partial tape that raises an error on any unknown read.

2.4 Halting preservation and reflection in PA

Induct on old transition count. At each boundary, configurations have the same head and entire tape and their states differ by ρ. The predecessor lemmas license the appropriate exceptional block; all other rows take one new step. Concatenating the nonempty blocks constructs exactly the deterministic new run.

A finite old halting run therefore gives a finite new halting run. For reflection, suppose the new machine halts by time S. If the old machine had not halted during its first S transitions, the block induction would construct a nonhalting new prefix of length at least S, a contradiction. This bounded argument is formalizable directly in PA and does not require an appeal to actual infinite objects. The local identities also show that no new halt occurs inside a block.

More precisely, if $n_{172,0}$ and $n_{211,0}$ count the indicated old rows in a halting computation, then

$$T_C = T_P + 6n_{172,0} + 2n_{211,0}.$$

The stated bounds follow. The complete old-state count is 209, the new count 208; the halt label is not included in either. This proves Theorem 1.1.

3 What the six incoming archives actually contain

The intake is pinned to ProveIt commit

`343da82d7384d10ec9fd1c17361fc923b2d1d117`.

The six named U209 ZIPs comprise five machine-bearing packets and one PDF-only supplement. The latter, whose name ends in `(1)`, declares the native and dense identities of the unqualified ConPA packet; it does not supply a sixth literal table.

Short name	Native hash prefix	Graph class	Source depth
ConPA	`89145587`	A	42
Compatibility	`4564c487`	B	41
Compatible Folds	`d0a155d3`	A	42
Composed	`8e058789`	A	42
Joint	`d0a155d3`	A	42
Joint (1)	PDF only	refers to A	—

Compatible Folds and Joint have byte-identical native and dense tables. The three byte-distinct class-A tables are exact state relabelings, not independent reductions that can simply be added. Canonical

breadth-first numbering from start 0, exploring reads 0 then 1 and preserving writes and directions, yields exactly two canonical graphs. Every vertex is reachable in these graphs, including the designated halt, so the canonicalization covers the full table.

Explicit witnesses fix all unlisted labels, start 0 and halt 254. From ConPA to Compatible Folds (and Joint), their nonidentity cycles are

$$(217\ 129\ 214)\,(215\ 204\ 216).$$

From ConPA to Composed, the nonidentity cycle is

$$(215\ 129\ 204\ 214\ 216).$$

Each of the 418 transition identities and all 210 vertices are checked. Consequently these class-A machines have identical head/tape histories step for step, modulo state names, on every common initial tape. Compatibility is not isomorphic to this class under the stated labelled, root-preserving convention.

All five packets have the same ordered 3,909 primitive operations and the same resolved ordinal source control flow. Class A has one common source-map byte string; Compatibility differs at exactly 720 source addresses, with no difference in ordered primitive effects or operands. All sources have maximum consecutive zero run six. The previous three savings remove native labels 180, 181 and 218. Class A uses keeper 163 for 180, whereas Compatibility uses keeper 216; both already remove 181 through 220 and 218 through 157. The new result removes a fourth label, 172, rather than double-counting an existing saving.

3.1 A second nonisomorphic 208 state variant

The same contextual fold also applies to the Plain U209 parent. Its native table is supplied in `machine/variant_plain/u208.native.tm` and its dense table in `machine/variant_plain/u208.tm`. The edits are again $116/0 : 0R172 \mapsto 0R211$ and $211/0 : 0R157 \mapsto 0L214$, followed by deletion of 172. The incoming-state census is checked separately: the intermediate state named 216 in the primary proof is named 163 here, while every entry to 116 and 214 still writes 1 and moves right. Thus the two marker lemmas, four endpoint identities and bounded simulation proof apply verbatim with the appropriate intermediate names (the primary state 163 corresponds to secondary state 216).

The secondary native SHA-256 is `8edb41dcfc693729868dd9ace2015a6da848e4152a282d4e21362dc33c551efd`; the dense SHA-256 is `b490b7dbe71b87143115dfc2be32cfbeac3bfe8e8bb5ee28119af249a3479c7a`. Its source is unchanged from Plain. A separate full replay proves all 5,084 contracts, covers all 416 rows, and uses 1,251,865 symbolic steps. Its complete 48,140-step blank startup agrees with the calibrated frame.

The inherited Plain-to-Folds, Plain-to-Composed and Plain-to-Joint bijections carry this fold to those three tables. The verifier constructs those derived tables and checks all rows under the explicit bijections; they are alternative names for the secondary variant. Rooted labelled canonicalization distinguishes the secondary from the primary variant. We therefore provide two distinct 208-state graph classes, not four independent reductions. The primary table remains the recommended artifact.

4 Independent final table verification

4.1 Universal interpreter contracts

The new local theorem already transports the exact parent's endpoint. A separate replay on the final 208-state table checks the interpreter without relying solely on that transport. There are 3,909 primitive occurrences: 1,558 increments, 1,175 guarded decrements, 80 forward branches, 1,095 backward branches and one halt. Splitting each decrement into zero and positive branches gives 5,084 cases and 5,083 nonhalt source edges.

These are a finite number of *symbolic derivations*, not tests of bounded register values. Their variables range over arbitrary natural registers and header gap, and arbitrary nonempty finite spare lists. The executable mathematics uses literal transitions, induction over unary scan lengths, and structural induction over finite lists after checking the one-block schema. Endpoints include the entire tape, head, source address and register frame. A positive decrement is represented with an arbitrary natural remainder plus one; zero and positive branches are disjoint and exhaustive.

The accepted primary-table replay produced

Guarded contracts	5,084
Literal rows covered	416 of 416
Symbolic transition steps	1,250,564
Working states checked	208

The runtime engine is self-authored verification infrastructure reused at exact file hashes, with assertions explicitly required. The supplied external Python programs are preserved only as evidence where selected; none is imported or executed by this release's replay.

This is not a proof-assistant kernel: soundness rests on the stated symbolic rules, their mathematical justification, and review of the ordinary Python implementation. Passing all literal rows is useful coverage evidence, but is not by itself a substitute for the quantified contracts or for the consistency calibration.

4.2 Exact startup and the interior source bridge

Fresh transition-by-transition replay verifies the complete 48,140-step blank prefix. The candidate, its parent, and the calibrated 212-state baseline have the same complete startup configuration and trace digest. The endpoint is native state 84 with head -2, header at 54, source ordinal 136 and source address `001100110101010`. In logical register order $(N, T, W, A, D, H, X, Y, L)$ the values are

$$(1, 0, 13, 0, 0, 1, 0, 0, 1).$$

There are 43 spare registers of value one and exactly 120 one-cells. The checker decodes the full finite support and verifies both zero exteriors. A separate source replay of 141 primitive steps from ordinal 106 and store $(1, 0, 4, 0, 0, 1, 0, 0, 1)$ reaches the displayed endpoint.

This is an interior axiom-builder continuation, not a clean program root. Its nonzero cache is an arithmetic value, not a theorem simply assumed valid. The retained protected-tail argument is needed to connect this configuration to the proof enumerator.

4.3 Independent reconstruction and failure checks

The current-parent audit reconstructs Compatibility from Research U211: eight closed rotations, the 218 fold, an eight-factor 720-leaf reassociation, three further rotations, a 171-entry repair, and the 180 fold. It checks the protected helper paths and ten finite local cases. The class-A audit independently reconstructs 30 closed rotations, the same entry repair, and its simultaneous three-state fold. These are baseline audits, not additional transformations in the new U208 construction.

The new relocatable verifier checks literal hashes, all state counts, totality, dense renamings, input-class witnesses, unchanged source, resolved control flow, startup, full frame, source bridge and the local four-case proof. Five negative controls independently corrupt the new read-zero direction, helper target, redirected 172 target, an ordinary row's existence, or an ordinary write. Each must be rejected. Finite checks also run under optimized Python; they use explicit exceptions, not removable assertions. The universal engine rejects optimized execution rather than silently dropping its obligations.

5 The inherited consistency endpoint

5.1 The exact unary theory

Theory U uses classical equality-free first-order logic over nonempty domains, with membership as its only nonlogical symbol. Put $\mathrm{Sub}(x,a) \equiv \forall z\,(z \in x \to z \in a)$. For every formula $\varphi(x)$ with no other free variable, its schema is

$$\forall a\,\exists b\,\big[\forall x\,(x \in b \to \varphi(x)) \;\wedge\; \forall x\,(\mathrm{Sub}(x,a) \to (\varphi(x) \to x \in b))\big].$$

Bound variables are freshened. The source a is not a permitted free matrix parameter. This is weak covering, not ambient extensionality and not an exact power filter.

The retained unary-theory proof gives interpretations in both directions with primitive-recursive finite-refutation translations. Its upper interpretation uses binary-bit membership and formula-specific finite comprehension in PA. The lower interpretation uses numerical representatives modulo coextensionality and saturated codes; induction is established before total successor existence. Parameters are bound inside legal unary matrices. The resulting ordinary mathematical statement is the PA-provable biconditional $\mathrm{Con}(\mathsf{U}) \leftrightarrow \mathrm{Con}(\mathrm{PA})$, not merely external semantic equiconsistency.

5.2 The program and protected initial tail

The machine searches the inherited numerical calculus $\mathcal{D}_U$. Its truth and falsity codes are 0 and 1; its proof steps include the logical and dominated propositional axioms, modus ponens, generalization and the gated covering bundle. This machine-facing cover presentation is parameter-rich and calibrated through unrestricted covering with global irreflexivity. Retained U214, U218 and U222 arguments handle translations, equality interpretation and small-code aliases. Its axiom strings are not literally the unary schema. The separate positive dyadic emitter in the unary package is not this program.

The startup calibration constructs nine theorem heads above opaque tail 4: seven logical instances, one dominated propositional instance and truth 0. Protection matters because the inherited pairing satisfies $4 = P(1,1)$ and $1 = P(1,0)$. The scheduler's prefix debt is at most two from candidate index one, leaving margin seven. A ghost list length handles $P(0,0) = 0$ without a numerical stack-length decoder. The protected tail does not become top or cache. The strong compiler above arbitrary underlying

stacks, together with cofinally successful candidate indices, gives completeness despite the finite initial cutoff.

These are inherited mathematical dependencies. The new local proof and runtime replay do not rederive every logical translation or export an arbitrary-proof translator. Their exact source files, original notices, licenses and nested archive identities are retained in the compact dependency selection. A theorem for the different K215 ZFC machine cannot certify this PA machine by substituting theory names.

5.3 Composition of the new and inherited theorems

The audited Compatibility parent has the retained ordinary calibration

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(P, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{U}).$$

Theorem 1.1 gives, without consistency assumptions,

$$\mathrm{PA} \vdash \mathrm{Halt}(C, \mathrm{blank}) \leftrightarrow \mathrm{Halt}(P, \mathrm{blank}).$$

Classical propositional reasoning inside PA, followed by the retained unary calibration, proves the endpoint in the abstract. Exact dense renaming transfers it to `machine/u208.tm`. Inheritance records proof provenance; it is not an extra axiom asserting that PA is consistent. Neither this argument nor the checks establish actual unconditional nonhalting.

6 Reproduction and evidence boundaries

The package is self-contained for the new transition and symbolic checks. From its extracted root run

```
python3 verify.py --output finite.json
python3 -O verify.py --output finite_optimized.json
python3 verify.py --full --output full.json
```

The first two commands need only Python's standard library. The full command needs ordinary Python with assertions enabled and replays the pinned self-authored unary/list-inductive engine; it does not run the supplied programs. Outputs may be placed outside the package to keep the sealed tree unchanged. The report can be rebuilt from `report/report.tex` using two runs of `pdflatex`, with the included transition-table TeX; PDF rebuilding is not needed for machine verification.

The release manifest binds all selected file bytes. The source-selection ledger separately records original archive members and original hashes: it does not pretend that omitted PDF copies or large regenerable case dumps remain present. The six input ZIP identities and the unary archive identity are retained even where duplicate archive bytes are not shipped. Original manifests are evidence of the originals, not a manifest of this compact derivative.

The report's complete dense table is generated from the machine-readable artifact. Final release checks extract the sealed ZIP to a fresh path, replay finite and full verification there, rebuild the PDF, compare extracted text and rendered page pixels, and record a standalone verification receipt. Every final PDF page is visually inspected.

Scope of the result. The construction demonstrates a specific 208-state machine below the requested threshold. It does not prove 208 minimal, classify all possible reductions, or treat failed bounded searches as impossibility results. Its genuinely new mathematical content is the contextual 172-to-211 fold; source and theory remain unchanged.

Sources and attribution

The five full U209 packets and PDF supplement come from Vladimir Reshetnikov's ProveIt incoming collection at the pinned commit above: `https://github.com/VladimirReshetnikov/ProveIt/tree/343da82d7384d10ec9fd1c17361fc923b2d1d117/docs/incoming`. The supplied Compatibility parent is credited for its earlier combined three-state reduction, proof-search program and retained calibration. The new result is not presented as a new proof of those inherited components. Full filenames and SHA-256 values are in `provenance/INPUTS.json`; the proof and license selection is recorded in the dependency ledger. The newly authored checker and the exact reused self-authored runtime-engine identities are distinguished from supplied code kept as text evidence.

Complete dense transition table 1 of 2

Working states are 0 through 207; halt is 208. Each entry is write, direction, next state. Start in state 0 with head at 0 on an all-zero tape.

State	Read 0	Read 1	State	Read 0	Read 1
0	1R92	1R0	52	0R67	1R68
1	0L1	1L2	53	0L40	1L53
2	0L3	1L2	54	0R22	1R54
3	0L5	1L2	55	0L11	0R50
4	1R6	0L4	56	0R60	0L16
5	0L9	1L2	57	0R69	1R70
6	0R6	0R10	58	0R61	1R71
7	0R189	1R159	59	0R54	1R59
8	0R14	1R13	60	0R49	0R25
9	0L15	1L2	61	0L28	1L72
10	0L16	0R10	62	0L117	1L30
11	0L26	1R128	63	0R73	1R63
12	0R19	1R20	64	0R97	1R150
13	0L89	1L31	65	1R184	1L53
14	0L11	0R22	66	0R144	1R145
15	0L23	1L2	67	0R76	1R208
16	1L1	0L1	68	0L196	1R31
17	0R24	0R32	69	0R41	1R62
18	0R98	1R113	70	0L89	1R78
19	0R28	1R29	71	0R55	1L79
20	0R30	1R31	72	0R80	1R81
21	0R24	0R59	73	1R0	1R63
22	0R33	1R22	74	0R74	1R82
23	0L34	1L2	75	0L145	1L141
24	0L35	0R0	76	0R85	1R86
25	0R36	1R25	77	0R20	1R38
26	0R37	1R38	78	0R89	1L132
27	0R39	0L40	79	0L108	1R56
28	0R41	1R42	80	0R49	0R33
29	0R43	1R44	81	0R91	0L42
30	0R45	1R46	82	0R177	1R77
31	0R21	1R47	83	0R96	1R12
32	0R48	1R32	84	0R88	1R97
33	0R32	1R33	85	0R206	1R21
34	0L74	1L2	86	0R116	1R100
35	1R35	1L1	87	0R173	1R166
36	0R50	1R36	88	0R102	1R103
37	0R17	1R111	89	0R80	1R79
38	0R13	1R52	90	0R101	1R195
39	0L11	0R36	91	0R105	1R106
40	0L4	1L53	92	0R65	0R0
41	0R24	0R25	93	0R27	1R107
42	0R13	0L16	94	0R182	0L207
43	0R55	1R56	95	1L109	1L95
44	0R57	1R58	96	0R19	1R110
45	0R24	0R54	97	0R60	1R60
46	0R60	1R61	98	0R68	1R135
47	0R62	0L16	99	0R93	1R108
48	0R25	1R48	100	0R105	0L207
49	0L35	0R63	101	0R117	1R118
50	0R16	1R50	102	0R71	1R58
51	0R147	1R143	103	0R57	1R119

Complete dense transition table 2 of 2

Working states are 0 through 207; halt is 208. Each entry is write, direction, next state. Start in state 0 with head at 0 on an all-zero tape.

State	Read 0	Read 1	State	Read 0	Read 1
104	0R21	1R120	156	0R150	1R112
105	0R21	0L10	157	0L186	1R96
106	0R61	1R42	158	0R167	1R178
107	0R121	1R8	159	0R19	1R170
108	0R101	1R122	160	0R164	1R186
109	0L207	0L95	161	0R159	1R96
110	0R124	1R104	162	0R125	1R164
111	0R115	1L72	163	0R97	1R162
112	0R194	1R180	164	0L189	1R142
113	0R160	1R183	165	0R146	1R171
114	0R148	1R146	166	0R198	1R200
115	0R11	1R129	167	0R161	1R143
116	0L131	0L1	168	0R148	1R87
117	0R39	1R72	169	0R161	1R96
118	0R61	1R132	170	0R180	1R122
119	0R61	1R133	171	0L192	1R60
120	0R70	0L16	172	0R145	1R88
121	0R14	1R132	173	0R152	1R201
122	0R21	1R134	174	0R140	1R84
123	0R151	1R188	175	0R174	1R75
124	0R137	1L20	176	0R126	1R168
125	0R130	1R179	177	0R138	1R116
126	0R203	1R199	178	0R97	1R51
127	0R75	1R175	179	0R167	1R51
128	0R75	1R205	180	0R17	1L131
129	0R202	1R157	181	0R143	1R149
130	0R12	1R141	182	0R76	1R116
131	0R30	1R69	183	0R106	1R124
132	0R39	1L124	184	1L40	1L109
133	0R153	0L16	185	0R174	1R199
134	0R154	0L16	186	0R181	1R182
135	0R88	1R136	187	0R66	1R204
136	0R196	1R140	188	0R140	1R139
137	0R24	0R48	189	0R84	1R190
138	0R99	1R37	190	0R157	1R185
139	0R171	1R148	191	0R64	1R7
140	0R96	1R159	192	0R150	1R127
141	0R174	1R165	193	0R18	1R94
142	0R12	1R159	194	0L197	1R172
143	0R88	1L165	195	0R104	1R96
144	0R88	1R158	196	0R90	1R88
145	0L152	1L163	197	0R199	1R156
146	0R84	1R88	198	0R96	1R123
147	0R159	1R12	199	0L194	1R194
148	0R83	1R143	200	0R201	1R197
149	0R144	1R155	201	0L123	1R167
150	0L66	1L141	202	0R196	1R28
151	0R150	1R169	203	0R163	1R192
152	0R159	1R114	204	0R146	1R176
153	0R97	1R97	205	0R161	1R193
154	0R49	0R32	206	0R45	1R100
155	0R191	1R187	207	0L4	0L207